Practical – 10

Aim-1: Thread-Pool Runner

With a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their ID and sleep briefly; shut down and await completion, observing thread reuse.

Code:

Main.java

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolRunner {

    public static void main(String[] args) throws InterruptedException {

        // Create a fixed thread pool of 3 threads
        ExecutorService service = Executors.newFixedThreadPool(3);

        // Submit 10 tasks
        for (int i = 1; i <= 10; i++) {

            final int taskId = i;

            service.submit(() -> {

                System.out.println("Task " + taskId +
                        " running on " +
                        Thread.currentThread().getName());

                try {
                    Thread.sleep(300);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }
```

Main.java

```java
public class ThreadPoolRunner {
    public static void main(String[] args) throws InterruptedException {
        // Submit 10 tasks
        for (int i = 1; i <= 10; i++) {

            final int taskId = i;

            service.submit(() -> {

                System.out.println("Task " + taskId +
                        " running on " +
                        Thread.currentThread().getName());

                try {
                    Thread.sleep(300);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        // Stop accepting new tasks
        service.shutdown();

        // Wait for all tasks to complete
        service.awaitTermination(10, TimeUnit.SECONDS);

        System.out.println("All tasks completed.");
    }
}
```

Output:

```
Task 2 running on pool-1-thread-2
Task 1 running on pool-1-thread-1
Task 3 running on pool-1-thread-3
Task 5 running on pool-1-thread-2
Task 4 running on pool-1-thread-3
Task 6 running on pool-1-thread-1
Task 8 running on pool-1-thread-2
Task 7 running on pool-1-thread-3
Task 9 running on pool-1-thread-1
Task 10 running on pool-1-thread-2
All tasks completed.
```

Learning Outcomes:

1. Understand task execution management using ExecutorService and thread pooling.
2. Observe thread reuse across multiple submitted Runnable tasks.

3. Manage thread pool lifecycle methods such as shutdown() and awaitTermination().

Identify performance gains of thread pooling compared to manual thread creation

Aim-2: Producer–Consumer Coordination

One thread adds numbers to a small shared buffer and another removes them, coordinated with wait() / notifyAll(); produce and consume items in order with none lost

Code:

Main.java

```
import java.util.LinkedList;
import java.util.Queue;

class SharedBuffer {

    private final Queue<Integer> buffer = new LinkedList<>();

    private final int capacity = 3;

    // Producer adds an item
    public synchronized void put(int value)
            throws InterruptedException {

        // Wait if buffer is full
        while (buffer.size() == capacity) {
            wait();
        }

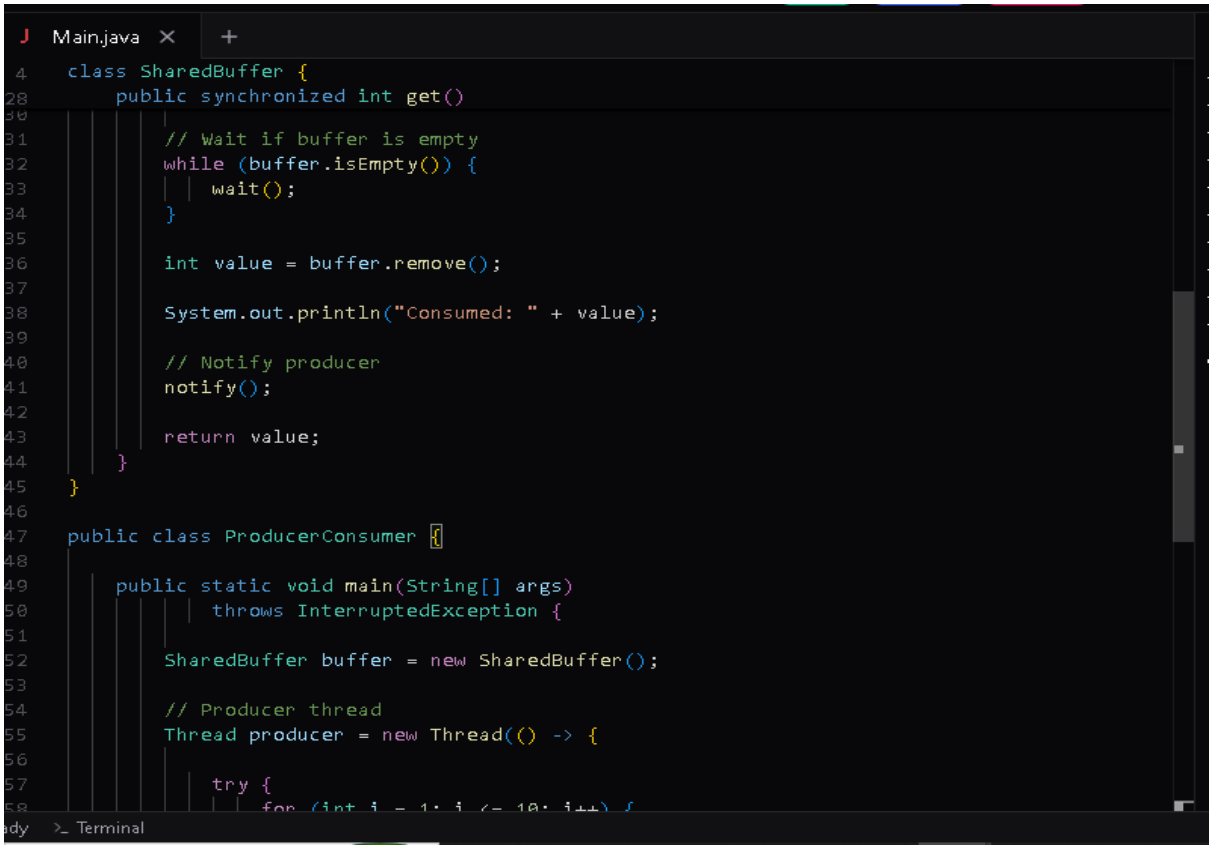
        buffer.add(value);

        System.out.println("Produced: " + value);

        // Notify consumer
        notify();
    }

    // Consumer removes an item
    public synchronized int get()
            throws InterruptedException {
```

Terminal



```
Main.java
class SharedBuffer {
    public synchronized int get()
        // Wait if buffer is empty
        while (buffer.isEmpty()) {
            wait();
        }

        int value = buffer.remove();

        System.out.println("Consumed: " + value);

        // Notify producer
        notify();

        return value;
    }
}

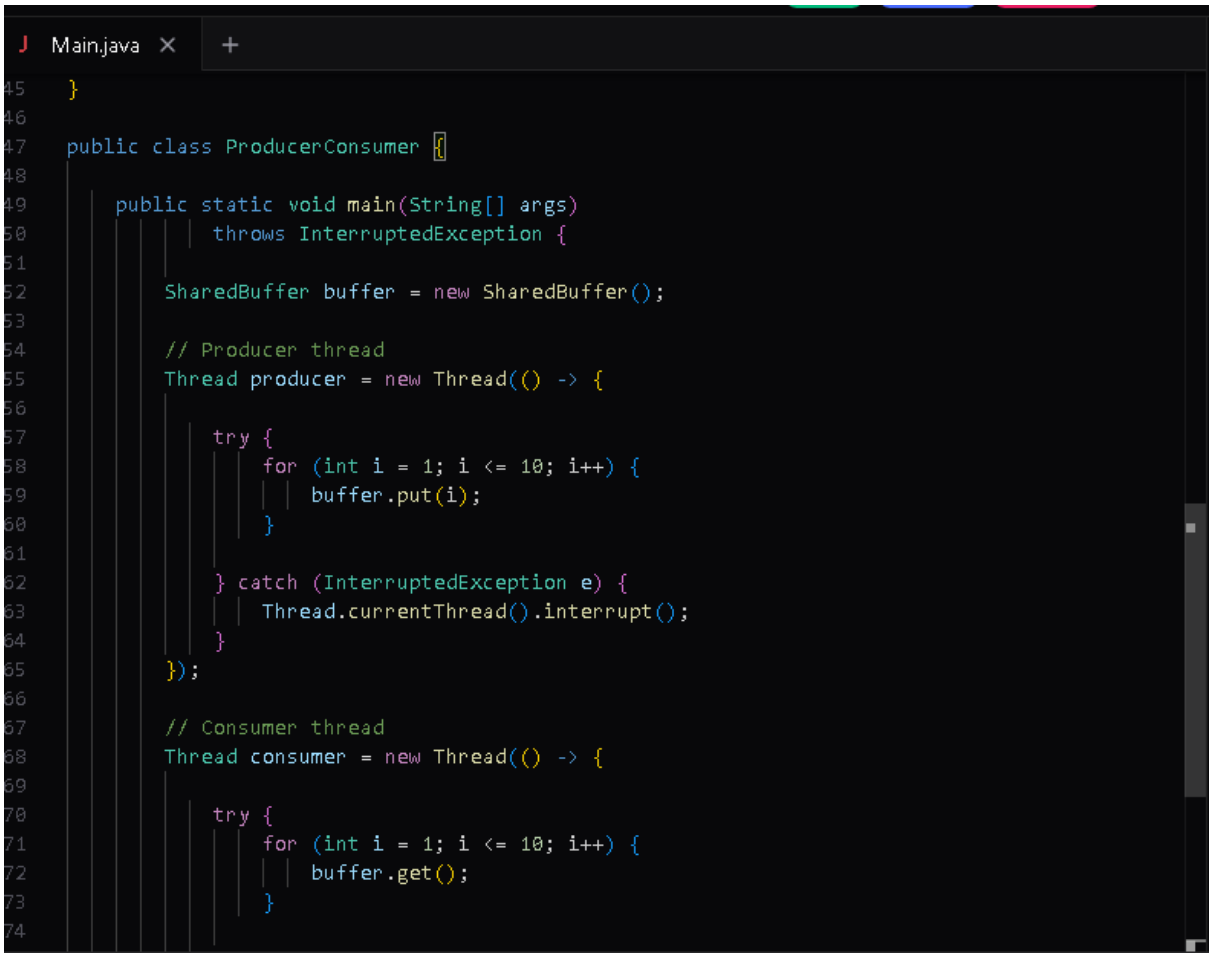
public class ProducerConsumer {

    public static void main(String[] args)
            throws InterruptedException {

        SharedBuffer buffer = new SharedBuffer();

        // Producer thread
        Thread producer = new Thread(() -> {

            try {
                for (int i = 1; i <= 10; i++) {
```



```
Main.java
}

public class ProducerConsumer {

    public static void main(String[] args)
            throws InterruptedException {

        SharedBuffer buffer = new SharedBuffer();

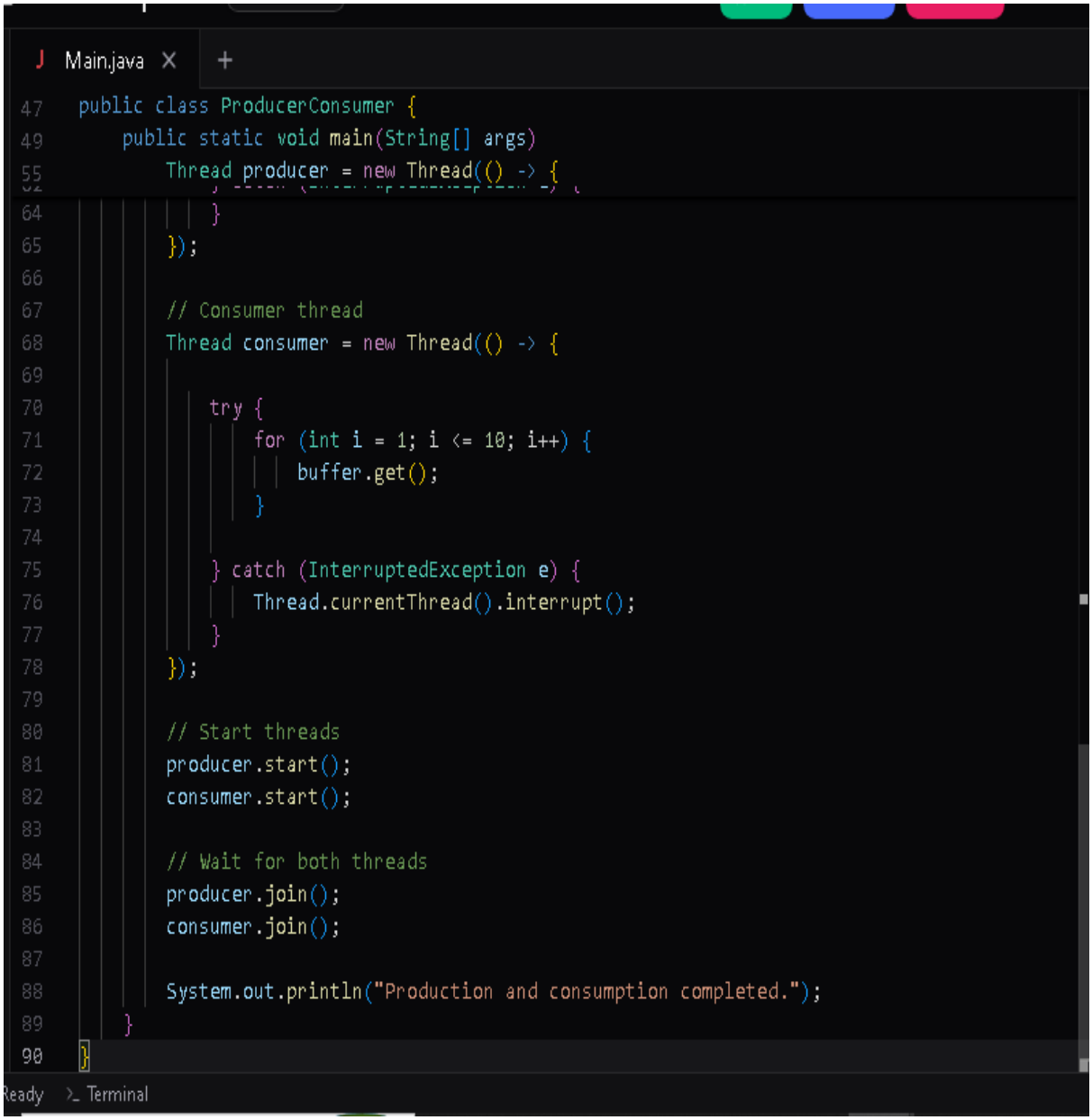
        // Producer thread
        Thread producer = new Thread(() -> {

            try {
                for (int i = 1; i <= 10; i++) {
                    buffer.put(i);
                }

            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        // Consumer thread
        Thread consumer = new Thread(() -> {

            try {
                for (int i = 1; i <= 10; i++) {
                    buffer.get();
                }
```



```
Main.java
47  public class ProducerConsumer {
49      public static void main(String[] args)
55          Thread producer = new Thread(() -> {
64              }
65          });
66
67          // Consumer thread
68          Thread consumer = new Thread(() -> {
69
70              try {
71                  for (int i = 1; i <= 10; i++) {
72                      buffer.get();
73                  }
74
75              } catch (InterruptedException e) {
76                  Thread.currentThread().interrupt();
77              }
78          });
79
80          // Start threads
81          producer.start();
82          consumer.start();
83
84          // Wait for both threads
85          producer.join();
86          consumer.join();
87
88          System.out.println("Production and consumption completed.");
89      }
90  }
Ready  >_ Terminal
```

Output:



Learning Outcomes:

1. Understand thread coordination using Java's monitor methods wait() and notifyAll().
2. Implement bounded buffer patterns for concurrent data sharing.
3. Prevent buffer underflow and overflow conditions in concurrent pipelines.

Ensure data integrity across producer and consumer threads without item loss.

Aim-3: (For Advanced Learners – home) Deadlock Reproduction & Resolution

Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock ordering strategy.

Code:

Main.java

```java
public class DeadlockDemo {

    private final Object lock1 = new Object();
    private final Object lock2 = new Object();

    // Both threads follow the same lock order
    private void task(String name) {

        synchronized (lock1) {

            System.out.println(name + " acquired lock1");

            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }

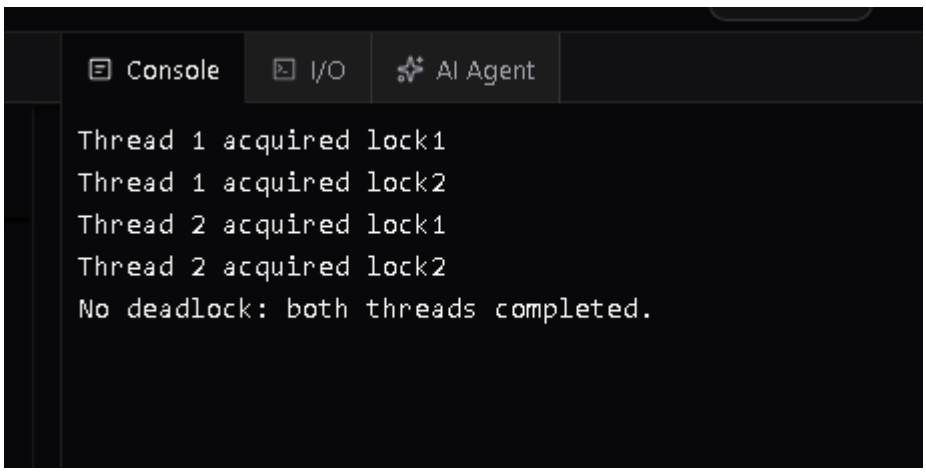
            synchronized (lock2) {

                System.out.println(name + " acquired lock2");
            }
        }
    }

    public static void main(String[] args)
            throws InterruptedException {

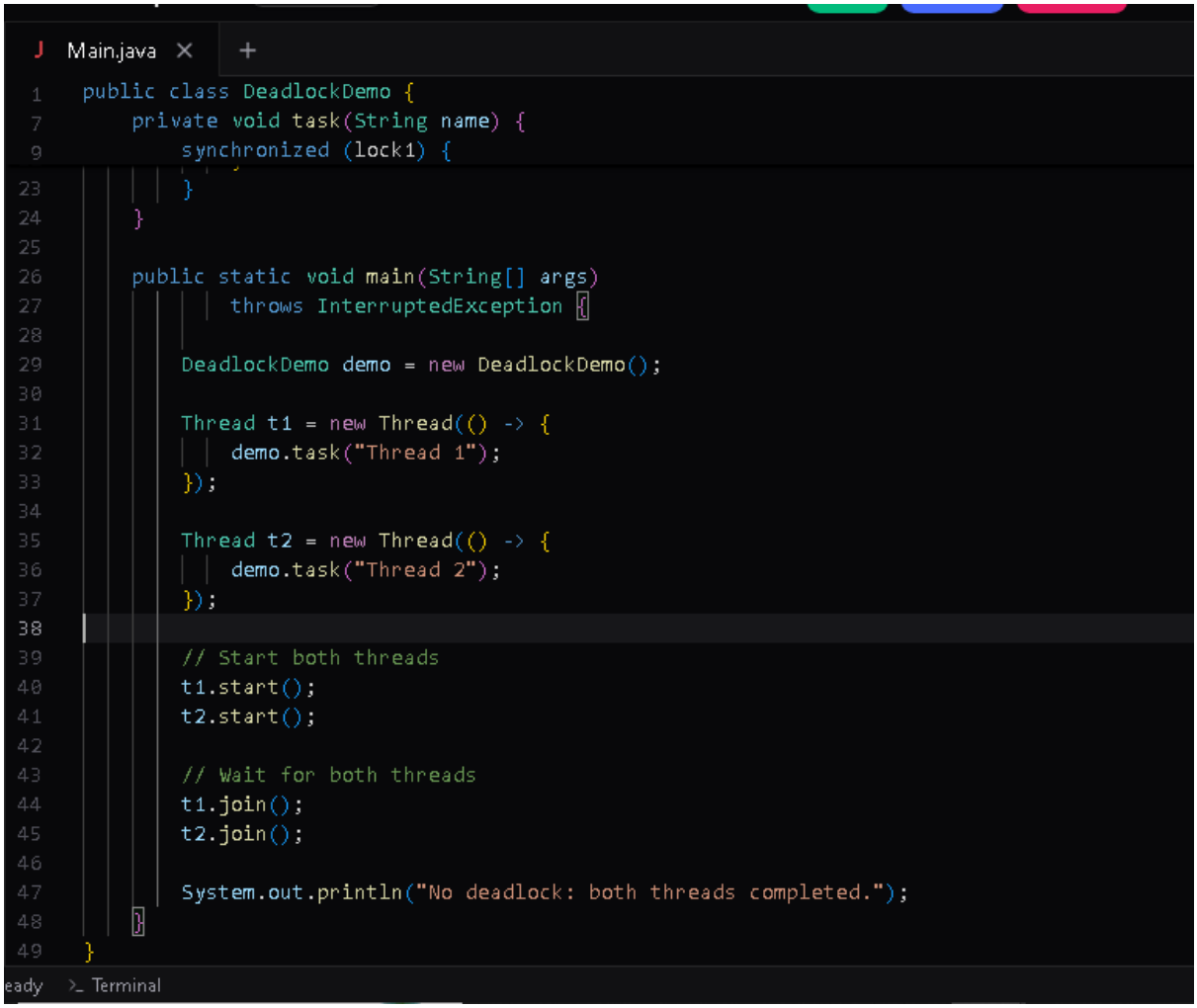
        DeadlockDemo demo = new DeadlockDemo();
```

Output:



Learning Outcomes:

1. Identify circular wait conditions that cause thread deadlocks in multithreaded systems.
2. Reproduce deadlock scenarios caused by inconsistent lock acquisition order across threads.
3. Apply global lock hierarchy and consistent lock ordering strategies to eliminate deadlocks.
4. Design thread-safe concurrent components that guarantee system execution progress.



Main.java

```
1   public class DeadlockDemo {
7       private void task(String name) {
9           synchronized (lock1) {
23          }
24      }
25
26      public static void main(String[] args)
27              throws InterruptedException {
28
29          DeadlockDemo demo = new DeadlockDemo();
30
31          Thread t1 = new Thread(() -> {
32              demo.task("Thread 1");
33          });
34
35          Thread t2 = new Thread(() -> {
36              demo.task("Thread 2");
37          });
38
39          // Start both threads
40          t1.start();
41          t2.start();
42
43          // Wait for both threads
44          t1.join();
45          t2.join();
46
47          System.out.println("No deadlock: both threads completed.");
48      }
49  }
```

eady >_ Terminal